

BOOTSTRAP5

Bootstrap 5:

Bootstrap is a powerful front-end framework for faster and easier web development. It includes HTML and CSS based design templates for creating common user interface components like forms, buttons, navigations, dropdowns, alerts, modals, tabs, accordions, carousels, tooltips, and so on. Bootstrap gives you ability to create flexible and responsive web layouts with much less efforts.

What You Can Do with Bootstrap

There are lot more things you can do with Bootstrap.

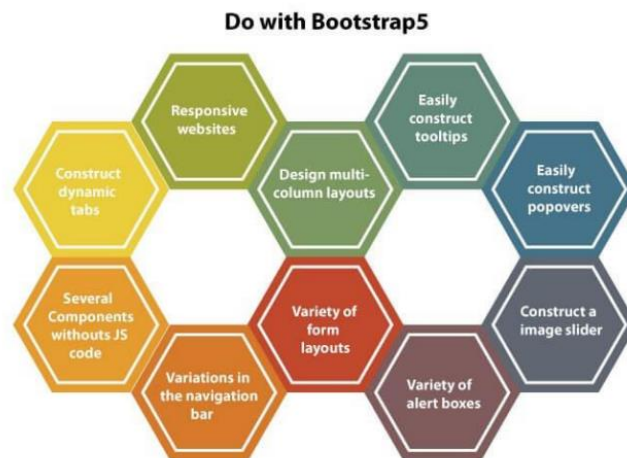
- You can easily create responsive websites.
- You can quickly create multi-column layout with pre-defined classes.
- You can quickly create different types of form layouts.
- You can quickly create different variation of navigation bar.
- You can easily create dynamic tabs to manage large amount of content.
- You can easily create tooltips and popovers to show hint text.
- You can easily create carousel or image slider to showcase your content.
- You can quickly create different types of alert boxes.

Advantages of Using Bootstrap

If you have had some experience with any front-end framework, you might be wondering what makes Bootstrap so special. Here are some advantages why one should opt for Bootstrap framework:

- Save lots of time — You can save lots of time and efforts using the Bootstrap predefined design templates and classes and concentrate on other development work. SE-3003 WEB ENGINEERING
- Responsive features — Using Bootstrap you can easily create responsive websites that appear more appropriately on different devices and screen resolutions without any change in markup.
- Consistent design — All Bootstrap components share the same design templates and styles through a central library, so the design and layout of your web pages will be consistent.
- Easy to use — Bootstrap is very easy to use. Anybody with the basic working knowledge of HTML, CSS and JavaScript can start development with Bootstrap.
- Compatible with browsers — Bootstrap is created with modern web browsers in mind and it is compatible with all modern browsers such as Chrome, Firefox, Safari, Internet Explorer, etc.
- Open Source — And the best part is, it is completely free to download and use.

Note: Bootstrap is responsive by default with a mobile first approach. Bootstrap 5 is the latest and most stable version of the Bootstrap. Bootstrap 5 is supported in all major modern browsers such Google Chrome, Firefox, Safari, Internet Explorer 10 and above, etc



Creating Containers with Bootstrap

Containers are the most basic layout element in Bootstrap and are required when using the grid system. Containers are basically used to wrap content with some padding. They are also used to align the content horizontally center on the page in case of fixed width layout. Bootstrap provides three different types containers:

- .container, which has a max-width at each responsive breakpoint.
- .container-fluid, which has 100% width at all breakpoints.
- .container-{breakpoint}, which has 100% width until the specified breakpoint.

Creating Responsive Fixed-width Containers

You can simply use the .container class to create a responsive, fixed-width container. The width of the container will change at different breakpoints or screen sizes,

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Bootstrap Responsive Fixed-width Containers</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet">
```

```

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"></script>

</head>

<body>

<div class="container">

<h1>This is a heading</h1>

<p>This is a paragraph of text.</p>

<p class="mt-3 text-info"><strong>Tip:</strong> Open the output in a new blank tab (Click the
arrow next to "Show Output" button) and resize the browser window to understand how the
Bootstrap
responsive grid system works.</p>

</div>

</body>

</html>

```

Creating Fluid Containers

You can use the .container-fluid class to create a full width container. The width of the fluid container will always be 100% irrespective of the devices or screen sizes.

Code:

```

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="utf-8">

<meta name="viewport" content="width=device-width, initial-scale=1">

<title>Bootstrap Fluid Containers</title>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet">

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"></script>

</head>

<body>

<div class="container-fluid">

```

```
<h1>This is a heading</h1>
<p>This is a paragraph of text.</p>
</div>
</body>
</html>
```

Adding Background and Borders to Containers

By default, container doesn't have any background-color or border or text color. But

if you need you can apply your own styles, or simply use the Bootstrap

background-color and border and text color utility classes to add backgroundcolor or border on them

Color:

The classes for text colors are: .text-muted, .text-primary, .text-success, .text-info, .text-warning, .text-danger, .text-secondary, .text-white, .text-dark, .text-body (default body color/often black) and .text-light;

Background Color:

The classes for background colors are: .bg-primary, .bg-success, .bg-info, .bg-warning, .bg-danger, .bg-secondary, .bg-dark and .bg-light.

All color, bg color and text color is implemented in following example:

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Set Background and Borders to Bootstrap Containers</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet">
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"></script>
</head>
<body>
<!-- Container with dark background and white text color -->
<div class="container bg-dark text-white border py-3 my-3">
```

```
<h1>This is a heading</h1>

<p>This is a paragraph of text.</p>

</div>

<!-- Container with light background -->

<div class="container bg-light py-3 my-3">

<h1>This is a heading</h1>

<p>This is a paragraph of text.</p>

</div>

<!-- Container with border -->

<div class="container border py-3 my-3">

<h1>This is a heading</h1>

<p>This is a paragraph of text.</p>

</div>

<p class="m-3"><strong>Note:</strong> The classes ".py-3", and ".my-3" are <a
href="/twitterbootstrap-tutorial/bootstrap-helper-classes.php" target="_top">spacing utility
classes</a> to add
padding, and margin of 1rem to the top and bottom sides of the element, respectively.</p>

</body>

</html>
```

Bootstrap Grid System

The Bootstrap 5 grid system is designed with flexbox and supports up to 12 columns across the page. If we don't want to use all 12 columns separately, you can combine them to make wider columns.

The grid system is responsive, and the columns will re-arrange themselves automatically based on screen size. It is not necessary to use all 12 available columns. Make sure the total is less than or equal to 12.

We can utilize Bootstrap's default grid classes to easily create layouts for various devices like mobile phones, tablets, laptops, and desktops.

For example, the .col-* classes can construct grid columns for especially small devices such as mobile phones in portrait mode, and the .col-sm-* classes for mobile phones in landscape mode.

Grid options

Bootstrap's grid system can adapt across all six default breakpoints, and any breakpoints you customize. The six default grid tiers are as follow:

1. Extra small (xs)
2. Small (sm)
3. Medium (md)
4. Large (lg)
5. Extra large (xl)
6. Extra extra large (xxl)

The following table shows the format of the bootstrap 5 grid systems.

Creating Two Column Layouts

The following example will show you how to create two column layouts for medium, large and extra large devices like tables, laptops and desktops etc. However, on mobile phones (screen width less than 768px), the columns will automatically become horizontal (2 rows, 1 column).

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Bootstrap Two Column Grid Layouts for Tablets and Desktops</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet">
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"></script>
<style>
/* Some custom styles to beautify this example */
.demo-content{
padding: 15px;
font-size: 18px;
background: #dbdfe5;
margin-bottom: 15px;
```

```
}  
.demo-content.bg-alt{  
background: #abb1b8;  
}  
</style>  
</head>  
<body>  
  <h2 class="text-center mt-3">Bootstrap Responsive Layout</h2>  
  <div class="text-center my-3">Open the output in a new blank tab (Click the arrow next to  
  "Show Output" button) and resize the browser window to understand how the Bootstrap responsive  
  grid system works.</div>  
  <div class="container">  
    <!--Row with two equal columns-->  
    <div class="row">  
      <div class="col-md-6">  
        <div class="demo-content">.col-md-6</div>  
      </div>  
      <div class="col-md-6">  
        <div class="demo-content bg-alt">.col-md-6</div>  
      </div>  
    </div>  
  
    <!--Row with two columns divided in 1:2 ratio-->  
    <div class="row">  
      <div class="col-md-4">  
        <div class="demo-content">.col-md-4</div>  
      </div>  
      <div class="col-md-8">  
        <div class="demo-content bg-alt">.col-md-8</div>  
      </div>  
    </div>
```

```

<!--Row with two columns divided in 1:3 ratio-->
<div class="row">
  <div class="col-md-3">
    <div class="demo-content">.col-md-3</div>
  </div>
  <div class="col-md-9">
    <div class="demo-content bg-alt">.col-md-9</div>
  </div>
</div>
</body>
</html>

```

Creating a Simple Table with Bootstrap

You can create tables with basic styling that has horizontal dividers and small cell padding (8px by default), by just adding the Bootstrap's class `.table` to the `<table>` element.

CODE:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Bootstrap Simple Table</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
    rel="stylesheet">
  <script
    src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"></script>
</head>
<body>
  <div class="m-4">
    <table class="table">

```



```
<thead>

<tr>

<th>#</th>

<th>First Name</th>

<th>Last Name</th>

<th>Email</th>

</tr>

</thead>

<tbody>

<tr>

<td>1</td>

<td>Clark</td>

<td>Kent</td>

<td>clarkkent@mail.com</td>

</tr>

<tr>

<td>2</td>

<td>Peter</td>

<td>Parker</td>

<td>peterparker@mail.com</td>

</tr>

<tr>

<td>3</td>

<td>John</td>

<td>Carter</td>

<td>johncarter@mail.com</td>

</tr>

</tbody>

</table>

</div>

</body>
```

</html>

A basic Bootstrap 5 table has a light padding and horizontal dividers. The

.table class adds basic styling to a table:

1. The .table-striped class adds zebra-stripes to a table
2. The .table-bordered class adds borders on all sides of the table and cells
3. The .table-hover class adds a hover effect (grey background color) on table rows
4. The .table-dark class adds a black background to the table

CODE:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="utf-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

```
<title>Bootstrap Dark Table</title>
```

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet">
```

```
<script
```

```
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"></script>
```

```
</head>
```

```
<body>
```

```
<div class="m-4">
```

```
<table class="table table-dark">
```

```
<thead>
```

```
<tr>
```

```
<th>#</th>
```

```
<th>First Name</th>
```

```
<th>Last Name</th>
```

```
<th>Email</th>
```

```
</tr>
```

```
</thead>
<tbody>
<tr>
<td>1</td>
<td>Clark</td>
<td>Kent</td>
<td>clarkkent@mail.com</td>
</tr>
<tr>
<td>2</td>
<td>Peter</td>
<td>Parker</td>
<td>peterparker@mail.com</td>
</tr>
<tr>
<td>3</td>
<td>John</td>
<td>Carter</td>
<td>johncarter@mail.com</td>
</tr>
</tbody>
</table>
</div>
</body>
</html>
```

Bootstrap Images

Documentation and examples for opting images into responsive behavior (so they never become larger than their parent elements) and add lightweight styles to them—all via classes. Images in Bootstrap are made responsive with `.img-fluid`. This applies `max-width: 100%;` and `height: auto;` to the image so that it scales with the parent element

```





```

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="utf-8">

<meta name="viewport" content="width=device-width, initial-scale=1">

<title>Bootstrap Image Styling</title>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet">

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"></script>

<style>

/* Custom style to add spacing between images */

img{

margin-right: 20px;

}

</style>

</head>

<body>

<div class="m-4">







</div>

</body>

</html>
```

Bootstrap Pagination

Pagination is the process of organizing content by dividing it into separate pages

If you have a web site with lots of pages, you may wish to add some sort of pagination to each page. To create a basic pagination, add the `.pagination` class to an `<ul>` element. Then add the `.page-item` to each `<li>` element and a `.page-link` class to each link inside `<li>`

```
<ul class="pagination">
  <li class="page-item"><a class="page-link" href="#">Previous</a></li>
  <li class="page-item"><a class="page-link" href="#">1</a></li>
  <li class="page-item"><a class="page-link" href="#">2</a></li>
  <li class="page-item"><a class="page-link" href="#">3</a></li>
  <li class="page-item"><a class="page-link" href="#">Next</a></li>
</ul>
```

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="utf-8">

<meta name="viewport" content="width=device-width, initial-scale=1">

<title>Bootstrap Pagination</title>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet">

</head>

<body>

<div class="m-4">

  <nav>

    <ul class="pagination">

      <li class="page-item"><a href="#" class="page-link">Previous</a></li>

      <li class="page-item"><a href="#" class="page-link">1</a></li>

      <li class="page-item"><a href="#" class="page-link">2</a></li>

      <li class="page-item"><a href="#" class="page-link">3</a></li>

      <li class="page-item"><a href="#" class="page-link">4</a></li>

      <li class="page-item"><a href="#" class="page-link">5</a></li>
```

```
<li class="page-item"><a href="#" class="page-link">Next</a></li>
</ul>
</nav>
</div>
</body>
</html>
```

Bootstrap Cards

Bootstrap card is a flexible and extensible content container. It includes options for headers and footers, a wide variety of content, contextual background colors, and powerful display options. The outer wrapper require the base class. card, whereas content can be placed inside the. card-body element

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Bootstrap Card</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
<div class="m-4">
<div class="card" style="width: 300px;">

<div class="card-body text-center">
<h5 class="card-title">Alice Liddel</h5>
<p class="card-text">Alice is a freelance web designer and developer based in
London. She is specialized in HTML5, CSS3, JavaScript, Bootstrap, etc.</p>
```

```
<a href="#" class="btn btn-primary">View Profile</a>

</div>

</div>

</div>

</body>

</html>
```

Task 1

Create a navbar using Bootstrap 5 where the navigation links are aligned to the right side of the header, with a custom background color and text color. Bg-color = #343a40 and text-color = #fff

Task 2

Create a registration form layout using Bootstrap5, and place it in a modal window that is vertically centred.

Task 3

Create a given layout using bootstrap 5 grid system

	Leslie Alexander leslie.alexander@example.com	Co-Founder / CEO Last seen 3h ago
	Michael Foster michael.foster@example.com	Co-Founder / CTO Last seen 3h ago
	Dries Vincent dries.vincent@example.com	Business Relations Online

task 4

Create a cart layout using Bootstrap 5 with a table structure. The table should have columns for the product image, product name, product price, and product quantity, which should include buttons to increase or decrease the quantity and update the total. The layout should be center-aligned, and at least 5 rows should be generated.

Task 5

Using the grid system, create a product page that contains three sections of products categorized by men's fashion clothes, women's fashion clothes, and kids' fashion clothes. Each category should have two rows of products, with each row containing four product cards on extra-large screens, three product cards on large screens, two product cards on medium screens, and a single product card on small screens. Each section should be aligned with equal spacing. The cards should display the product name, price, and a details button, with high-quality images. The cards should be of equal size

JavaScript

Introduction to JavaScript:

JavaScript often abbreviated as JS, is a programming language that is one of the core technologies of the World Wide Web, alongside HTML and CSS. As of 2022, 98% of websites use JavaScript on the client side for webpage behavior, often incorporating third-party libraries. All major web browsers have dedicated JavaScript engine to execute the code on users' devices. JavaScript is a high-level, often just-in-time compiled language that conforms to the ECMAScript standard. It has dynamic typing, prototype-based object-orientation, and first-class functions. It is multi-paradigm, supporting event-driven, functional, and imperative programming styles. It has application programming, interfaces (APIs) for working with text, dates, regular expressions, standard data structures, and the Document Object Model (DOM). The ECMAScript standard does not include any input/output (I/O), such as networking, storage, or graphics facilities. In practice, the web browser or other runtime system provides JavaScript APIs for I/O. JavaScript engines were originally used only in web browsers, but are now core components of some servers and a variety of applications. The most popular runtime system for this usage is Node.js. Although Java and JavaScript are similar in name, syntax, and respective standard libraries, the two languages are distinct and differ greatly in design.

Scripting language vs programming language

Programming Language	Scripting Language
A programming language is an organized way of communicating with a computer.	A scripting language is a programming language that supports scripts.
Traditional programming is based on low level languages.	Scripting prefers high level languages.
The traditional programming languages such as C, C++, and Java are compiled.	Perl, Python, JavaScript, and other languages used for scripting are interpreted and do not require the compilation step.
General programming leads to closed software applications.	Scripting promotes open projects and is used for web applications.
More code needs to be written.	Less coding is required in scripting.

Variable declaration using VAR, LET and CONST:

	let	var	const
Scope	Block	Function	Block
Declaring same name	Not possible	Possible	Not possible
Updating value	Possible	Possible	Not possible (Property is updatable)
Use before declaration	Error	Undefined	Error

Types of JavaScript Operators

There are different types of JavaScript operators: [➤ Arithmetic Operators](#)

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

Example:

```
<p id="demo"></p>
```

```
<script>
```

```
let a = 3;
```

```
let x = (100 + 50) * a;
```

```
document.getElementById("demo").innerHTML = x;
```

```
</script>
```

• Assignment Operators

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

Assignment operators assign values to JavaScript variables. The Addition Assignment Operator (+=) adds a value to a variable.

Example:

```
<p id="demo"></p>
```

```
<script>
```

```
var x = 10;
```

```
x += 5;
```

```
document.getElementById("demo").innerHTML = x;
```

```
</script>
```

Comparison Operators

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

Logical Operators

Operator	Description
&&	logical and
	logical or
!	logical not

Type Operators

Operator	Description
typeof	Returns the type of a variable
instanceof	Returns true if an object is an instance of an object type

Conditional statements & decision making

IF ELSE FAMILY:

Conditional statements are used to perform different actions based on different conditions.

Conditional Statements Very often when you write code, you want to perform different actions for different decisions. You can use conditional statements in your code to do this. In JavaScript we have the following conditional statements:

Use if to specify a block of code to be executed, if a specified condition is true Use else to specify a block of code to be executed, if the same condition is false Use else if to specify a new condition to test, if the first condition is false Use switch to specify many alternative blocks of code to be executed

The if Statement

Use the if statement to specify a block of JavaScript code to be executed if a condition is true.

Syntax

```
if (condition) {  
  // block of code to be executed if the condition is true  
}
```

```
if (hour < 18) {  
  greeting = "Good day";  
}
```

Switch Statement

The switch statement is used to perform different actions based on different conditions.

```
switch(expression) {  
  case x:  
    // code block  
    break;  
  case y:  
    // code block  
    break;  
  default:  
    // code block  
}
```

```
switch (new Date().getDay()) {  
  case 0:  
    day = "Sunday";  
    break;  
  case 1:  
    day = "Monday";  
    break;  
  case 2:  
    day = "Tuesday";  
    break;  
  case 3:  
    day = "Wednesday";  
    break;  
  case 4:  
    day = "Thursday";
```

```
break;

case 5:

day = "Friday";

break;

case 6:

day = "Saturday";

}
```

Functions

A JavaScript function is a block of code designed to perform a particular task. A JavaScript function is executed when "something" invokes it (calls it).

Types of function

User define function

Built-in function

builtin functions are

```
alert();
```

```
prompt();
```

```
parseInt();
```

```
parseFloat();
```

```
Number();
```

syntax:

```
<script>
```

```
alert("this is an altdddr");
```

```
</script>
```

Task 1

Task 01: Write a program in which all types of Data types of JS will be used using LET, VAR, CONST and NOTHING variable initialization.

Task 2

Create a program for admission where the criteria are tested based on the given input. If the percentage of the intermediate exam is greater than 80%, the student should be eligible to attempt the test. Otherwise, the program should display the respective message. If the student has a percentage greater than 65% in the test, they will be eligible for admission; otherwise, the program should show the respective message. The program must check that only positive numbers are entered in the prompts.

Task 3

Create a program that checks if today is your birthday. After giving the prompt, it should check the date. If the current date matches the birthday, it should display a 'Happy Birthday' message. If the current date has passed the birthday, it should display a message saying, 'It will be next year.' If the given prompt contains a date that is in the future but hasn't arrived yet, it should display a message saying, 'Excited to wish you a happy birthday' the program should be validate from incorrect data